

ISAT341 Final Project: Image Classification with Scikit-Learn

Objective

In this final project, YOU will build and evaluate an image classification model in Python using Scikit-Learn inside Anaconda/Jupyter Notebook.

The main focus is on:

- loading and exploring an image or image-like dataset,
- training one Scikit-Learn classification model,
- evaluating the model with standard classification metrics,
- explaining the results clearly.

Tools

YOU may use:

- Python
- Jupyter Notebook through Anaconda
- Scikit-Learn
- NumPy
- Matplotlib
- Pandas, optional

Dataset Choice

A classical dataset is the built-in Scikit-Learn Digits dataset. This dataset contains small images of handwritten digits from 0 to 9.

- Number of classes: 10
- Class labels: digits 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9
- Each image is represented as an 8 by 8 grid of pixel values

Student choice option: YOU may instead choose another image or image-like classification dataset that interests you, as long as it can be loaded and used in Jupyter Notebook with Scikit-Learn. Briefly explain what the dataset contains and what classes you are trying to predict.

Task 1: Load and Explore the Dataset

Load the default Digits dataset, or load your own image/image-like dataset. Briefly describe what YOU are trying to classify.

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
import numpy as np

# Option A: recommended default dataset
digits = load_digits()

X = digits.data # Flattened image data: one row per image
y = digits.target # Correct labels
images = digits.images # 8 by 8 image arrays

print("Feature matrix shape:", X.shape)
print("Target shape:", y.shape)
print("Class names:", digits.target_names)
```

If YOU use your own dataset: replace `X`, `y`, and `images` with your own data. Make sure your data can be used by Scikit-Learn.

Display at least 6 sample images or examples.

```
plt.figure(figsize=(10, 4))

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(f"Label: {y[i]}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Note: If YOU choose a dataset with a different image size, adjust the display code as needed.

Task 2: Split the Data and Train One Model

Split the data into training and testing sets. Then choose **one** Scikit-Learn classification algorithm and train it.

Recommended model choices:

- Random Forest Classifier
- Support Vector Machine, also called SVM
- K-Nearest Neighbors, also called KNN
- Logistic Regression

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)

```

Note: The example uses Random Forest. YOU may replace it with SVM, KNN, or Logistic Regression if YOU prefer.

Task 3: Evaluate the Model

Make predictions on the test set. Then report the accuracy, classification report, and confusion matrix.

```

from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import ConfusionMatrixDisplay

# Make predictions
y_pred = model.predict(X_test)

# Evaluate performance
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
print(classification_report(y_test, y_pred))

# Confusion matrix
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion Matrix")
plt.show()

```

Task 4: Show Prediction Examples

Display at least 6 test images or examples with their predicted labels and actual labels.

```

plt.figure(figsize=(10, 4))

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(X_test[i].reshape(8, 8), cmap="gray")
    plt.title(f"Pred: {y_pred[i]} | Actual: {y_test[i]}")
    plt.axis("off")

plt.tight_layout()

```

```
plt.show()
```

Note: The reshape line above is for the default Digits dataset. If YOU use a different dataset, change the reshape size or display method.

Task 5: Short Reflection

Write one short paragraph answering the following:

- Which dataset and algorithm did YOU use?
- What was the accuracy?
- Based on the confusion matrix, where did the model do well or make mistakes?
- Overall, did the model perform well? Explain briefly.

Optional Extension

For extra analysis, try **one** of the following:

- change one hyperparameter, such as `n_estimators`, `n_neighbors`, or `kernel`,
- compare your model to a second Scikit-Learn classifier,
- use scaling with `StandardScaler` for SVM, KNN, or Logistic Regression,
- use cross-validation with `cross_val_score`.

Final Submission

Submit one Jupyter Notebook that includes:

- dataset loading and sample images or examples,
- a brief description of the dataset and classes,
- train/test split,
- one trained Scikit-Learn model,
- accuracy, classification report, and confusion matrix,
- at least 6 prediction examples,
- one short reflection paragraph.

Grading Rubric

Category	Marks
Dataset selected, loaded, and explored correctly, including sample images/examples	20
Training/testing split completed correctly	10
One Scikit-Learn model chosen and trained correctly	25
Accuracy, classification report, and confusion matrix included	25
Prediction examples displayed clearly	10
Short reflection explains the results clearly	10
Total	100